

OWNER REFERENCE

How Star Player of the Week is built

A living document — updated as the app changes.

A plain reference for talking about this app with full confidence — what it's built with, how the data is organized, what's protected against, and how a change actually goes live.

01 WHAT IT'S BUILT WITH

FRONTEND

React + TypeScript

Built with Vite. Styled with Tailwind CSS on a small custom set of colors, spacing, and type sizes.

BACKEND

Firebase

No custom server, no Cloud Functions, and no sign-in of any kind — not even an invisible one. Firebase just stores the data.

DATABASE

Cloud Firestore

Data is stored as documents in named groups called "collections," not tables. The app gets live updates automatically, without asking for them.

HOSTING

Firebase Hosting

The site is hosted on Google's servers and deployed from the command line. There's no separate server to maintain.

UI PARTS

Radix UI + custom

Radix supplies accessible dialogs and toasts. Everything else — buttons, fields, badges — is built by hand to match the brand.

TESTING

Vitest + a Firestore test copy

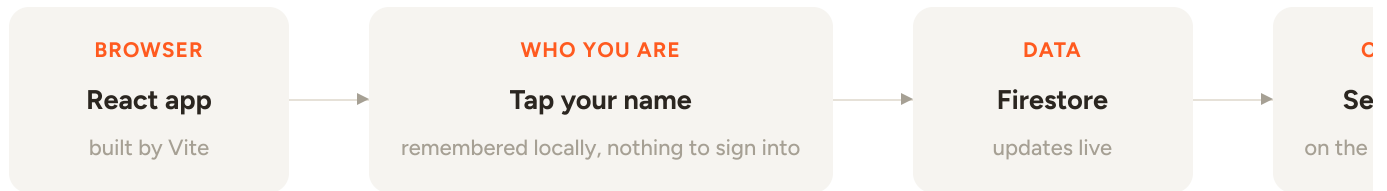
Unit tests check plain logic. A separate test suite runs the real security rules against a local test copy of the database, to prove access rules actually work.

CODE STORAGE

GitHub

One main branch. Every change is saved as a commit before it goes live.

02 HOW A REQUEST FLOWS THROUGH IT



Nobody signs in — not with a password, not even invisibly. Tapping a name on the roster just writes a note to your own browser's storage saying that's who you are; there's no account behind it and no server checking that the tap belongs to the right person. That was a deliberate trade: once the app stopped holding anything worth protecting that way, keeping any form of sign-in around stopped paying for itself.

There's no backend server deciding permissions — the app talks to the database directly, and a separate file called **Security Rules** decides what a given request is actually allowed to do. That matters less than it used to: with no identity behind any request, these rules mostly check the **SHAPE** of a write (is this really just a name? did the count only move by one?) rather than **WHO** is making it. The one thing they still gate on timing rather than identity is the vote count — see "What's protected" below.

One name on the roster is still different: whoever's browser has claimed **KG** sees the controls to run the session — opens and closes voting, pauses a week, calls the reveal. They don't vote themselves. There's no role to assign and no account to promote; the app's own screen decides this purely by checking which name you've locally claimed, the same as everything else about "who you are" here.

03 HOW THE DATA IS ORGANIZED

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
sotw_profiles	The roster — each teammate's name, plus their photo if they've added one. Nothing else	Everyone
sotw_claims	Which name has been tapped. Written once, when someone taps it, and never changed after — only ever deleted and re-created	Everyone
sotw_meta/settings	Is voting open, which week it is, is the week paused, has it been revealed, who won	Everyone
sotw_voters	That a person has voted this week. There is no field here for who they voted for	Everyone

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
sotw_tally	The running count of votes for each candidate — numbers only, no names attached to them	Everyone, once voting closes
sotw_stat_status	Each person's own "stats up" or "stats down" for the current week	Everyone
sotw_weekly_activity	That someone received at least one vote in a given week — what the streak badges are counted from	Everyone

Every name starts with `sotw_` — a simple prefix so this data can share a database with something else later without clashing.

There is no collection for "who I voted for", and that absence is the design. Your own pick is written to your browser's storage and nowhere else, so the link between a person and the name they chose never reaches the database at all. The cost is small and deliberate: clear your browser data and the app forgets which card to tick for you — the vote itself still stands, you just can't change it from a browser that no longer remembers it.

04 WHAT'S PROTECTED

✓ **Nobody can learn who voted for whom.** It is never recorded — not for teammates, not for KG, not for anyone reading the raw database. Two things get stored: that a person voted, and a running count for each candidate, and neither field can be stretched to carry the other (the rules reject any extra field on either doc). This is a real guarantee, not a convention — it holds regardless of who's asking, because nobody is ever checked.

✓ **The count is unreadable while voting is open.** Whether it's KG, a teammate, or someone poking the database directly, the read is refused until voting closes — so nobody can watch a number tick up and work out who just voted. Once it closes, anyone can technically read it; the app's own screen still only shows it to whoever's claimed KG.

What's *not* a database-level guarantee any more, now that this app dropped every form of sign-in: "one name, one person" and "only KG runs the session" are conventions the app's own screens follow, not things the rules can verify — there's no identity left to check them against. A name being claimed by *somebody* can't be un-claimed out from under them by a second tap (the record is create-only), but nothing stops a browser from claiming a different name that isn't really theirs by going around the app's UI. For a small team with nothing valuable riding on it, that was judged an acceptable trade for having no sign-in step at all.

One further accepted trade-off, unrelated to identity: because the server holds no record of your pick, it can't check a vote being taken back against the vote you actually cast — it can only hold each change to a single point up or down. Someone hand-crafting requests could nudge a rival's count by one. Closing that would mean storing who voted for whom, which is precisely the thing this design refuses to do, so the one-point limit is where the line sits.

05 SHIPPING A CHANGE

Usually shipped together — the app and the rules it depends on:

```
# app code, and the rules it depends on, in one go
firebase deploy --only hosting,firestore:rules
```

They can still be deployed separately (`--only hosting` , or `--only firestore:rules` on their own) when that's genuinely useful — publishing a rules change ahead of the app code that needs it, say. But there's no longer a standing reason to keep them apart by default: with no accounts and no roles left to guard, a routine app update can't accidentally change who's allowed to do what, because nothing here is gated on who's asking any more.

06 WHAT'S HAPPENED SO FAR

Rebuilt from one big file

The app used to be a single large HTML file. It's now a proper React + TypeScript codebase, easier to maintain and grow. The old version is being phased out.

Security check-up

A full review found and fixed real gaps — including a way anyone could inflate vote counts.

Made the session controls self-serve

Adding, renaming and removing people, opening and closing voting, and recovering from a stuck state can now all be done from the app — no direct database edit needed.

Stats up or down, streaks, photos

Everyone says whether their stats are up before they vote, and only "up" goes on the poll. Photos, a badge for anyone who's received votes three weeks running, and a "pause this week" switch for holidays landed alongside it.

Stripped back to just the vote

Accounts and the Blacfox Dollars wallet are gone, and every screen that wasn't the vote went with them. Open the app, tap your name, vote — that's the whole app now.

Dropped the last trace of sign-in

Even the invisible kind — tapping your name used to hand your browser an anonymous Firebase identity behind the scenes, purely so the rules could enforce one name per browser. With nothing left worth protecting that way, that step is gone too, and "who you are" now just lives in your own browser's storage. The trade: one name per person is a convention the app follows now, not something the database can verify.

